

MODUL PRAKTIKUM 3

NULL SAFETY DAN PEMROGRAMAN ASINKRON

3.1. Topik Bahasan

Modul ini dilaksanakan pada pertemuan ke-3 sesuai Rencana Pembelajaran Semester mata kuliah Pemrograman IV dan membebani CPMK-02, dengan cakupan topik sebagai berikut:

- Deklarasi tipe nullable dan non-nullable.
- Operator null-aware `?.`, `??`, `??=`, serta *keyword* `late`.
- Penanganan *error* menggunakan `try`, `catch`, `on`, dan `finally`.
- Konsep *Future* dan mekanisme eksekusi asinkron.
- Penggunaan `async` dan `await` pada pengambilan data pengiriman yang tertunda.

3.2. Capaian Pembelajaran

Mahasiswa mampu menulis program yang aman terhadap nilai null dan menangani proses yang berjalan secara asinkron (CPMK-02).

3.3. Indikator Ketercapaian Pembelajaran

Indikator penilaian pertemuan ke-3 menurut Rencana Pembelajaran Semester adalah:

- Ketepatan penerapan null safety
- Ketepatan penanganan error
- Ketepatan penggunaan fungsi asinkron

Indikator tersebut dinyatakan tercapai apabila bukti berikut terlihat pada hasil praktikum:

- Program berjalan tanpa error null pada seluruh skenario uji.
- Error tertangani dengan pesan yang jelas dan tidak menghentikan program secara paksa.
- Data hasil proses asinkron tampil setelah proses selesai, bukan sebelum proses dimulai.
- Penggunaan `await` dan penanganan *Future* tepat sesuai alur program.

3.4. Teori Pendukung

Null Safety

Sejak Dart versi 2.12 seluruh tipe bersifat non-nullable secara bawaan. Artinya sebuah variabel bertipe `String` dijamin selalu berisi teks dan tidak pernah bernilai null. Apabila suatu variabel memang boleh kosong, tipenya harus ditandai dengan tanda tanya,

misalnya `String?`. Jaminan ini diperiksa oleh *compiler* sehingga error akses nilai null terdeteksi sebelum program dijalankan, bukan setelah aplikasi dipakai pengguna.

Pada aplikasi logistik, sejumlah data memang bersifat opsional: catatan penerima, koordinat lokasi pengantaran, atau waktu barang diterima. Data semacam itu tepat dimodelkan sebagai tipe nullable, sedangkan data wajib seperti nomor resi dan kota tujuan dimodelkan sebagai tipe non-nullable.

Operator Null-Aware

- `?.` mengakses anggota hanya bila nilainya tidak null, dan menghasilkan null bila nilainya null.
- `??` memberikan nilai pengganti ketika operand di sebelah kirinya bernilai null.
- `??=` menetapkan nilai hanya bila variabel masih bernilai null.
- `!` memaksa compiler menganggap nilai tidak null. Operator ini hanya boleh digunakan bila benar-benar yakin, karena akan memunculkan error saat program berjalan bila ternyata nilainya null.
- `late` menunda pengisian nilai variabel non-nullable hingga saat pertama kali digunakan.

Kode Program 3.1 Penggunaan operator null-aware

```
String? catatanPenerima;
DateTime? waktuDiterima;

final panjangCatatan = catatanPenerima?.length ?? 0;
catatanPenerima ??= 'Tidak ada catatan';

final status = waktuDiterima == null ? 'Dalam perjalanan' : 'Diterima';
```

Tabel 4. Operator null-aware pada bahasa Dart

Operator	Fungsi	Contoh
<code>?.</code>	Mengakses anggota hanya bila nilainya tidak null	<code>catatan?.length</code>
<code>??</code>	Memberi nilai pengganti ketika operand kiri null	<code>catatan ?? "kosong"</code>
<code>??=</code>	Mengisi variabel hanya bila masih null	<code>catatan ??= "kosong"</code>
<code>!</code>	Memaksa nilai dianggap tidak null	<code>waktu!.toIso8601String()</code>
<code>late</code>	Menunda pengisian variabel non-nullable	<code>late final Database db;</code>

Penanganan Error

Error pada Dart dibedakan menjadi Error yang menandakan kekeliruan pemrograman dan Exception yang menandakan kondisi tidak normal yang wajar terjadi, misalnya koneksi terputus atau data tidak ditemukan. Blok `try` membungkus kode yang berpotensi gagal, blok `on` menangkap jenis *exception* tertentu, blok `catch` menangkap *exception* secara umum beserta *stack trace*-nya, dan blok `finally` selalu dijalankan baik terjadi error maupun tidak.

Exception sebaiknya ditangkap pada tingkat yang mampu mengambil keputusan, lalu disampaikan kepada pengguna dalam bahasa yang dapat dipahami. Menampilkan pesan teknis mentah kepada pengguna akhir bukan merupakan penanganan error yang memadai.

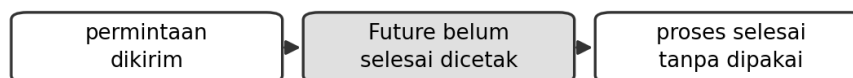
Pemrograman Asinkron dan Future

Dart mengeksekusi kode pada satu *thread* yang dikelola melalui *event loop*. Operasi yang memakan waktu, seperti pemanggilan layanan web atau pembacaan basis data, tidak boleh menghentikan thread tersebut karena antarmuka akan membeku. Operasi semacam itu dijalankan secara asinkron dan hasilnya diwakili oleh objek Future.

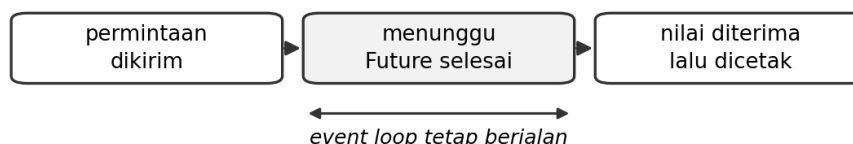
Sebuah Future berada pada salah satu dari tiga keadaan: belum selesai, selesai dengan nilai, atau selesai dengan error. Keyword `await` menunggu Future selesai lalu mengambil nilainya, dan hanya boleh digunakan di dalam fungsi yang ditandai `async`. Fungsi `async` selalu mengembalikan Future, sehingga fungsi yang tidak mengembalikan nilai ditulis dengan tipe `return Future<void>`.

Ketika beberapa operasi asinkron tidak saling bergantung, `Future.wait` dapat dipakai untuk menjalankannya secara bersamaan sehingga total waktu tunggu jauh lebih singkat dibandingkan menunggu satu per satu.

Sinkron (tanpa await)



Asinkron (dengan await)



Gambar 3. Perbedaan urutan eksekusi tanpa dan dengan await

3.5. Alat Pendukung

Perangkat keras, perangkat lunak, dan bahan yang diperlukan pada modul ini adalah:

- Flutter SDK dan Dart SDK.
- Visual Studio Code beserta ekstensi Flutter dan Dart.
- Terminal PowerShell.
- Repositori GitHub pribadi praktikan (satu repositori untuk seluruh modul praktikum).

3.6. Langkah-langkah Praktikum

A. Penyiapan

1. Buka proyek `silog_app` melalui menu `File > Open Folder` pada Visual Studio Code, lalu buka terminal terintegrasi melalui menu `Terminal > New Terminal`. Buat cabang kerja modul ini pada terminal dengan perintah `git checkout -b praktikum/modul-03`, kemudian pastikan cabang aktif melalui `git branch`.
2. Buat dua berkas di dalam direktori latihan melalui panel Explorer: klik kanan direktori latihan, pilih `New File`, lalu ketik `modul03_nullsafety.dart`; ulangi langkah yang sama untuk `modul03_async.dart`.

B. Latihan 1: Null Safety

1. Ketik kode berikut pada berkas `latihan/modul03_nullsafety.dart`, lalu simpan melalui `Ctrl+S`.

Kode Program 3.2 Penanganan data pengiriman yang bersifat opsional

```
class DataKiriman {  
  final String resi;           // wajib  
  final String kotaTujuan;     // wajib  
  final String? catatan;       // opsional  
  final DateTime? waktuTerima; // opsional  
  
  DataKiriman({  
    required this.resi,  
    required this.kotaTujuan,  
    this.catatan,  
    this.waktuTerima,  
  });  
}  
  
String ringkasan(DataKiriman k) {  
  final catatan = k.catatan ?? '(tanpa catatan)';  
  final status = k.waktuTerima == null  
    ? 'Dalam perjalanan'  
    : 'Diterima pada ${k.waktuTerima!.toIso8601String()}';  
  return '${k.resi} | ${k.kotaTujuan} | $status | $catatan';  
}  
  
void main() {  
  final daftar = <DataKiriman>[  
    DataKiriman(resi: 'SLG-001', kotaTujuan: 'Bandung'),  
    DataKiriman(  
      resi: 'SLG-002',  
      kotaTujuan: 'Surabaya',  
      catatan: 'Titipkan ke satpam',  
      waktuTerima: DateTime(2026, 9, 12, 10, 30),  
    ),  
  ];  
  
  for (final k in daftar) {  
    print(ringkasan(k));  
  }  
}
```

2. Jalankan program melalui `dart run latihan/modul03_nullsafety.dart`.

3. Hapus tanda tanya pada deklarasi String? catatan, amati pesan error dari compiler, kemudian kembalikan seperti semula dan catat penjelasannya pada jurnal.

C. Latihan 2: Penanganan Error

1. Tambahkan kode berikut pada berkas latihan/modul03_nullsafety.dart, panggil fungsi ujiPenanganan() dari main(), simpan melalui Ctrl+S, lalu jalankan ulang melalui dart run latihan/modul03_nullsafety.dart.

Kode Program 3.3 Penanganan error dengan try-catch

```
class ResiTidakDitemukan implements Exception {
  final String resi;
  ResiTidakDitemukan(this.resi);
  @override
  String toString() => 'Resi $resi tidak ditemukan pada basis data.';
}

final Map<String, String> basisResi = {
  'SLG-001': 'Bandung',
  'SLG-002': 'Surabaya',
};

String cariKota(String resi) {
  final kota = basisResi[resi];
  if (kota == null) {
    throw ResiTidakDitemukan(resi);
  }
  return kota;
}

void ujiPenanganan() {
  for (final resi in ['SLG-001', 'SLG-999']) {
    try {
      print('$resi -> ${cariKota(resi)}');
    } on ResiTidakDitemukan catch (e) {
      print('Peringatan: $e');
    } catch (e, s) {
      print('Kesalahan tidak terduga: $e');
      print(s);
    } finally {
      print('Pencarian $resi selesai.');
    }
  }
}
```

D. Latihan 3: Future, async, dan await

1. Ketik kode berikut pada berkas latihan/modul03_async.dart, lalu simpan melalui Ctrl+S.

Kode Program 3.4 Simulasi pengambilan data pelacakan secara asinkron

```
Future<String> ambilStatusKiriman(String resi) async {
  // Simulasi jeda jaringan selama dua detik
  await Future.delayed(const Duration(seconds: 2));

  if (!resi.startsWith('SLG-')) {
    throw FormatException('Format resi tidak sah: $resi');
  }
}
```

```

    }
    return 'Paket $resi sedang dalam perjalanan menuju gudang transit.';
}

Future<double> ambilOngkir(String resi) async {
    await Future.delayed(const Duration(seconds: 1));
    return 105400;
}

Future<void> main() async {
    print('1. Permintaan data dikirim...');

    try {
        final status = await ambilStatusKiriman('SLG-002');
        print('2. $status');

        final ongkir = await ambilOngkir('SLG-002');
        print('3. Ongkos kirim: Rp${ongkir.toStringAsFixed(0)}');
    } on FormatException catch (e) {
        print('Kesalahan format: ${e.message}');
    } catch (e) {
        print('Gagal mengambil data: $e');
    }

    print('4. Proses selesai.');
```

2. Jalankan program melalui `dart run latihan/modul03_async.dart`, lalu catat urutan kemunculan baris *output* beserta jeda waktunya.
3. Hapus keyword `await` pada baris pemanggilan `ambilStatusKiriman`, jalankan ulang, lalu jelaskan mengapa yang tercetak adalah objek Instance of 'Future<String>'.
4. Kembalikan keyword `await`, lalu ubah argumen menjadi 'XYZ-002' untuk menguji jalur penanganan error.

E. Latihan 4: Eksekusi Paralel dengan Future.wait

Kode Program 3.5 Menjalankan beberapa operasi asinkron secara bersamaan

```

Future<void> bandingkanWaktu() async {
    final mulai = DateTime.now();

    final hasil = await Future.wait([
        ambilStatusKiriman('SLG-001'),
        ambilOngkir('SLG-001'),
    ]);

    final durasi = DateTime.now().difference(mulai);
    print('Status : ${hasil[0]}');
    print('Ongkir : ${hasil[1]}');
    print('Durasi total: ${durasi.inMilliseconds} ms');
```

1. Panggil fungsi `bandingkanWaktu()` dari `main()`, simpan melalui `Ctrl+S`, jalankan program melalui `dart run latihan/modul03_async.dart`, lalu bandingkan durasinya dengan pemanggilan berurutan pada Latihan 3.

2. Tuliskan kesimpulan mengenai kapan `Future.wait` layak digunakan dan kapan tidak.

F. Tugas Praktikum

1. Kembangkan fungsi `ambilStatusKiriman` sehingga membaca data dari sebuah Map berisi minimal lima resi, dan melemparkan `ResiTidakDitemukan` apabila resi tidak terdaftar.
2. Susun fungsi `pantauBanyakResi(List<String> daftarResi)` yang memanggil fungsi tersebut untuk seluruh resi secara bersamaan, menampilkan hasil yang berhasil, dan mencatat resi yang gagal tanpa menghentikan proses.
3. Lampirkan tangkapan layar output program untuk skenario seluruh resi sah dan skenario terdapat resi tidak sah.

G. Luaran yang Dinilai

Bentuk penilaian modul ini adalah tugas praktikum dengan kriteria program bebas error null dan kesesuaian penggunaan `async/await`. Jurnal praktikum individu diunggah ke LMS ulbi paling lambat 5x24 jam setelah sesi praktikum. Agar kriteria tersebut dapat dinilai, praktikan menyerahkan luaran berikut:

- Berkas `modul03_nullsafety.dart` dan `modul03_async.dart`.
- Bukti program bebas error null berupa tangkapan layar dart `analyze` tanpa temuan.
- Tangkapan layar output untuk skenario resi sah dan skenario resi tidak sah.
- Penjelasan tertulis mengenai urutan kemunculan baris output pada program asinkron, beserta perbandingan durasi pemanggilan berurutan dan `Future.wait`.

3.7. Panduan Pengumpulan Jurnal Hasil Kegiatan Praktikum

Tugas praktikum modul ini bersifat individu. Setiap praktikan menyusun dan mengunggah sendiri jurnalnya ke LMS ULBI paling lambat 5 x 24 jam setelah sesi praktikum berakhir, berupa berkas PDF bernama `NIM_Nama_Modul-XX.pdf`, disertai tautan repositori GitHub pribadi pada cabang `praktikum/modul-XX`.

Sistematika jurnal:

1. Identitas: nama, NIM, kelas, nomor dan judul modul, serta tanggal praktikum.
2. Tujuan praktikum.
3. Alat dan bahan yang digunakan beserta versinya.
4. Langkah kerja, disertai tangkapan layar tiap tahap penting.
5. Kode program beserta penjelasan singkatnya.
6. Hasil eksekusi dan analisisnya, termasuk error yang ditemui beserta penyelesaiannya.
7. Jawaban seluruh butir Tugas Praktikum.
8. Kesimpulan dan kendala.

9. Tautan repositori beserta tangkapan layar riwayat *commit*.

Tangkapan layar wajib memperlihatkan jendela *browser* atau emulator secara utuh beserta kode programnya, dan memuat nama serta NIM praktikan. Praktikan yang tidak mengumpulkan jurnal memperoleh nilai nol; keterlambatan dan ketidaksesuaian hasil lainnya dikenai pengurangan nilai sesuai kebijakan dosen pengampu.

Seluruh butir pada sub bab Luaran yang Dinilai wajib dilampirkan pada jurnal.

3.8. Hasil Capaian Pembelajaran Praktikum

Bentuk dan kriteria penilaian mengikuti Rencana Pembelajaran Semester pertemuan ke-3: Tugas praktikum: program bebas error null dan kesesuaian penggunaan *async/await*. Jurnal praktikum individu diunggah ke LMS ULBI paling lambat 5x24 jam setelah sesi praktikum.

Tugas praktikum modul ini bersifat individu dan dinilai dari jurnal praktikum masing-masing praktikan.

Bobotnya 2% dari nilai akhir mata kuliah pada komponen evaluasi Praktikum (Aktivitas Partisipatif) sebesar 15% dari nilai akhir, diukur dari kelengkapan dan mutu luaran pada sub bab Luaran yang Dinilai terhadap indikator penilaian modul ini.